

In-Class Warm-Up Lab: Thinking Like a Programmer

Intro to Programming (Python)

1 Part 1: What is an Algorithm?

Activity: Write steps to make a sandwich.

Questions

- Are your steps clear enough for a computer?
- What happens if a step is missing?

Check Yourself

- Can someone else follow your steps exactly?

Key Idea: Algorithms must be clear and precise.

English Sandwich Algorithm

- Step 1: Grab bread, meat, sauce, lettuce, and cheese
- Step 2: Place the bread on the counter
- Step 3: Put on your sauce of choice
- Step 4: Place the cheese on top
- Step 5: Place the lettuce and meat on top of the cheese,
- Step 6: Put the other piece of bread on top

These instructions are relatively clear but vague

2 Part 2: Step Counting

```
numbers = [5 , 3 , 8 , 2 , 9]
target = 9
for num in numbers :
    if num == target :
        print ( " Found ! " )
```

Questions

- How many times does the loop run? (**5 times, once per element in the list**)
- What if the target is the first element? (**The loop finds the target instantly but still continues to loop to check the other list items**)
- What if the target is not present? (**The loop still ends after 5, but “found” is not printed**)

Check Yourself

- Can you explain the difference between best and worst case? (**I did above!**)

3 Part 3: Nested Work

Questions

- How many times does print run? (**((3 times for i and 3 times for j (3*3 =9)) n*n = N^2))**)

- What happens when the list grows? (The list becomes larger but n numbers in list square to create the amount of total checks ($n_8 = 8 \times 2 = 16$))

Part 4: Memory Usage

Questions

- Which code stores more data? (**neither code currently stores more data**)
- Why does storing data matter? (**Storing data heavily affects memory usage, it is important for later uses in code and application**)

Check Yourself

- Can you identify when extra memory is used? (**If code creates extra lists, copies, strings, or new data structures, that code stores *more* data.**)

5 Part 5: Thinking Before Coding

Questions

- What steps would you write before coding? (**below**)
- Why not directly start coding? (**You need to think about the security, memory usage, efficiency, what language to use, and many more planning features before you can jump into a project**)

Check Yourself

- Can you explain your logic in plain English?

Part 6: Two Ways to Solve a Problem

Questions:

- Which method feels faster? Stop counting (**as it does not check all combinations of the list**)
- Which uses extra memory? (**Technically, nested loops may take more memory when calling them, or using a list function such as “append”**) (**(Neither innately use more memory but nested loops will normally be used in higher memory usages)**)
- Why might we choose one over the other? (If you need all the data in a list, Nested loops are helpful, if you are searching for a single element, stop counting is much more efficient)

7 Part 7: Predict the Output

```
numbers = [2 , 4 , 6]
count = 0
for num in numbers :
    count += 1
print ( count )
```

Questions

- What will be printed? (**“3”**) (**there are 3 elements in the list**)
- How many times does the loop run? (**3 times, (stop counting depends on the amount of items in the list)**)

8 Part 8: Find the Mistake

```
numbers = [3 , 1 , 5 , 2]
max_val = 0
for num in numbers :
    if num > max_val :
        max_val = num
print ( max_val )
```

Questions

- What is wrong with this logic? **(it does not account for negative numbers, if the list had negative numbers, the value would print “0”)**
- What happens if all values are negative? **(above)**

Check Yourself

- Can you fix the issue? (Write if max_val = numbers[0]) This will check each item and assume it is the lowest, accounting for the negative numbers in the list)

9 Part 9: Reduce the Work

Questions

- Which version is easier to understand? Part 7 code, **(counting)**
- Which avoids unnecessary steps? **(Counting) Yet If statements can be used to do more specific tasks)**

Check Yourself

- Can you simplify your own code? **(Possibly, also leveraging tools to help simplify and enhance readability.)**

Key Idea: Simpler code is often better.

10 Part 10: Build Your Own Algorithm

Questions

- Are your steps complete? **(Yes)**
- Can someone else follow them? **(Probably)**

Check Yourself

- Did you avoid skipping steps? **(I don't think so)**

Key Idea: Clear thinking leads to good programs.

11 Part 11: Real-Life Connection

Questions

- Why is a catalog system faster? **(Catalogs organize information, without manually having to check all data)**
- Can you think of similar examples? **(search by name, rather than numbers)**

Check Yourself

- Can you connect this idea to programming? **(yes)**

Key Idea: Organization reduces work.

12 Part 12: Challenge

```
numbers = [2 , 8 , 1 , 10 , 4]
```

Count numbers greater than 5

Questions

- What steps will you follow?

```
numbers = [2, 8, 1, 10, 4]
```

```
Count = 0
```

```
    for num in numbers:
```

```
        if num > 5:
```

```
            Count += 1
```

```
print(count)
```

- How many times will the loop run? (5, but it depends on the number of elements)

Check Yourself

- Did you write steps before coding? (**no...**) (**I should have...**) (**I did in my head...**)

13 Reflection

1. What is an algorithm? (**A precise, step-by-step set of instructions to accomplish a set of tasks**)
2. What does "more work" mean in a program? (**More sorting, more data, larger code, more memory**)
3. What did you learn about thinking before coding? (**Identify what type of algorithm you need to use as a developer, memory, language, user interface**)